

SRCS : Systèmes Répartis Client/Serveur

TD 3 : Application répartie

Objectifs pédagogiques :

- Protocole applicatif
- Gestion des erreurs
- Protocole avec/sans état

Exercice 1 – Service de calculatrice

En utilisant un protocole de transport fiable (TCP), on souhaite réaliser un serveur qui implante les fonctionnalités d'une calculatrice élémentaire fournissant quatre opérations arithmétiques (+ - * /). Des clients doivent donc pouvoir se connecter à distance et demander au serveur la réalisation d'une opération. Le serveur leur répond en fournissant le résultat ou un compte rendu d'erreur.

Question 1

Définir un protocole applicatif (i.e. un ensemble de messages et leur enchaînement) aussi simple que possible implantant cette spécification.

Question 2

Recenser les cas d'erreur pouvant se produire avec ce protocole applicatif et proposer un comportement à adopter lorsque ces erreurs surviennent (on ne s'intéresse ici ni aux erreurs de transport du style message perdu, ni aux pannes de machines).

Question 3

En utilisant les primitives fournies ci-dessous donner le pseudo-code du serveur et le pseudo-code d'un client demandant la réalisation de l'opération $8.6 * 1.75$:

- `ouvrirCx(serveur)` : demande d'ouverture de connexion vers le serveur
- `attendreCx()` : attente bloquante et acceptation d'une demande de connexion
- `envoyer(données)` : envoi de données
- `recevoir()` : attente bloquante et réception de données
- `fermerCx()` : fermeture de connexion

Question 4

Le serveur est-il avec ou sans état ?

Remarque : on se place au niveau des primitives définies à la question précédente. On ne s'intéresse pas au protocole de transport sous-jacent.

Question 5

Si vous avez répondu sans état à la question précédente, donner un exemple de serveur calculette avec état. Réciproquement, si vous avez répondu avec état, donner un exemple de serveur calculette sans état.

Question 6

On souhaite que plusieurs clients puissent se connecter simultanément sur le serveur. Cela pose-t-il un problème particulier ? Modifier le pseudo-code du serveur pour prendre en compte ce cas.

Question 7

On souhaite maintenant que les clients puissent enchaîner plusieurs demandes d'opérations au sein d'une même connexion. Proposer deux solutions pour cela (une en modifiant le protocole précédent et une sans modification du protocole).

Exercice 2 – Producteurs/Consommateurs répartis

Dans cet exercice on s'intéresse à des processus client communiquant par l'intermédiaire d'un schéma producteur/consommateur. Dans ce schéma, les communications entre processus se font par l'intermédiaire d'un tampon où sont stockés les messages émis par les producteurs. Ces messages peuvent ensuite être retirés par les consommateurs. Les processus producteurs et consommateurs sont situés sur des sites distincts connectés par des lignes bidirectionnelles. On suppose que la communication s'effectue sans perte de message.

On se place ici dans le cas où les messages sont de taille fixe, et l'on considérera que le tampon est une table composée de N entrées, chaque entrée ayant la taille d'un message. L'API du tampon propose deux opérations **non atomiques** :

- `put(msg)` qui ajoute dans le tampon un message
- `msg get()` qui retire un message du tampon et renvoie ce message comme résultat

Ci-dessous, une implantation centralisée de la gestion du tampon :

<pre> 67 Mutex verrou ← InitialiserMutex(); 68 Déposer(msg) 69 begin 70 Lock(verrou); 71 Attendre(nbMsg < N); 72 tampon.put(msg); 73 tampon.nbMsg ++; 74 Unlock(verrou); </pre>	<pre> 75 Retirer() 76 begin 77 Lock(verrou); 78 Attendre(nbMsg > 0); 79 msg ← tampon.get(); 80 tampon.nbMsg --; 81 Unlock(verrou); 82 return msg; </pre>
---	--

Question 1

Quels sont les problèmes posés par la répartition des ressources ?

Question 2

En considérant un processus producteur et un processus consommateur, indiquer les avantages et les inconvénients d'une implémentation sur

- deux sites : un site pour le consommateur et un site pour le producteur
- trois sites : le tampon a un site dédié

Question 3

On se place dans le cas d'une implémentation sur deux sites. Pourquoi ne programme-t-on pas **deposer** et **retirer** comme dans la version centralisée ?

Question 4

Donner les algorithmes de **deposer** et **retirer**.

Question 5

Quels sont les nouveaux problèmes qui se posent si on lève l'hypothèse de fiabilité des communications ?

Question 6

Comment peut-on gérer la panne d'un site ?

Question 7

Donner les algorithmes de **deposer** et **retirer** dans le cas d'une implémentation sur trois sites.

Question 8

Proposer une solution résistant à la panne du site contenant le tampon.